



Phase 3: Distributed Data (Partitioning and Replication)

TU856

CMPU4003: Advanced Databases

Conor Egan
C22466756

School of Computer Science
Technological University, Dublin

14/12/2025

Contents

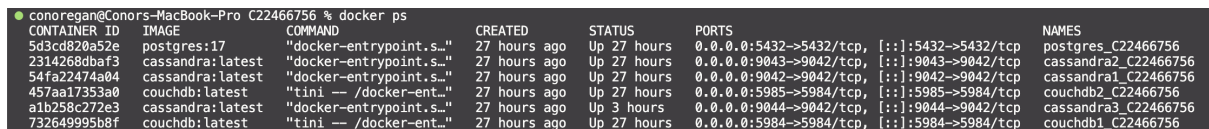
1. Data Sources & Setup	1
2. PostgreSQL Partitioning	2
3. Cassandra Partitioning and Replication	9
4. CouchDB Partitioning and Replication	15
5. Comparative Evaluation.....	19

1. Data Sources & Setup

This project implements a distributed data architecture comprising three distinct database technologies orchestrated via Docker. The environment allows for the comparative analysis of consistency models (ACID vs BASE) and partitioning strategies.

Environment Details

The infrastructure was deployed using Docker Desktop on macOS. A custom docker-compose file was configured to spin up a 6-container cluster on a shared bridge network. This included PostgreSQL version 17, a 3-node Cassandra cluster to test quorum consensus, and a 2-node CouchDB cluster to demonstrate Master-Master replication. All database artifacts are suffixed with the student identifier C22466756 to ensure traceability.



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5d3cd820a52e	postgres:17	"docker-entrypoint.s..."	27 hours ago	Up 27 hours	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	postgres_C22466756
2314268dbaf3	cassandra:latest	"docker-entrypoint.s..."	27 hours ago	Up 27 hours	0.0.0.0:9043->9042/tcp, [::]:9043->9042/tcp	cassandra2_C22466756
54fa22474a04	cassandra:latest	"docker-entrypoint.s..."	27 hours ago	Up 27 hours	0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp	cassandra1_C22466756
457aa17353a0	couchdb:latest	"tini -- /docker-ent..."	27 hours ago	Up 27 hours	0.0.0.0:5985->5984/tcp, [::]:5985->5984/tcp	couchdb2_C22466756
alb258c272e3	cassandra:latest	"docker-entrypoint.s..."	27 hours ago	Up 3 hours	0.0.0.0:9044->9042/tcp, [::]:9044->9042/tcp	cassandra3_C22466756
732649995b8f	couchdb:latest	"tini -- /docker-ent..."	27 hours ago	Up 27 hours	0.0.0.0:5984->5984/tcp, [::]:5984->5984/tcp	couchdb1_C22466756

Figure 1: docker ps - 6 containers running with Student ID C22466756

PostgreSQL Setup

The PostgreSQL instance from Phase 2 was reused to serve as the source of truth for this phase. The dw_lite_C22466756 schema was used, specifically the fact_sales table and associated dimensions. The existing Docker volume was mounted to preserve the Phase 2 Data Warehouse. To support the partitioning tests in Section 2, the fact_sales table was duplicated and restructured into three specific schemas for Range, List, and Hash partitioning strategies.

Cassandra Setup

To demonstrate high availability and partition tolerance, a subset of the PostgreSQL sales data was migrated to a 3-node Cassandra cluster. A query was run against PostgreSQL to extract granular sales data which was mapped to a new denormalized schema suitable for column-family storage. This included the use of Collection Datatypes to demonstrate complex data modelling without joins. Two keyspaces were created using NetworkTopologyStrategy to explicitly compare the resilience of Replication Factor 1 versus Replication Factor 3.

CouchDB Setup

A 2-node CouchDB cluster was configured to demonstrate multi-master replication and offline-sync capabilities. Synthetic JSON documents representing Inventory and Sales Events were generated to mirror the semantic structure of the relational data but adapted for a schema-less document model. Unlike Phase 2, this phase utilised the _replicator database to configure continuous, bi-directional sync between the nodes enabling conflict resolution testing.

2. PostgreSQL Partitioning

Objective

The primary objective of this section was to implement and critically evaluate three distinct horizontal partitioning strategies (Range, List, and Hash) within the PostgreSQL data warehouse environment. The aim was to determine how physical data separation influences the Query Planner's decision-making process, specifically regarding partition pruning and I/O reduction, compared to non-partitioned baseline tables.

2.1 Implementation

The implementation utilised the `dw_lite_C22466756` schema and the `fact_sales` table populated in Phase 2. Three separate table structures were designed to test specific query patterns.

Range Partitioning was implemented to optimise time-series analysis, a common requirement in data warehousing. The table `fact_sales_range_C22466756` was partitioned on the `date_key` column. Specific partitions were created for the years 2024 and 2025 using the `FOR VALUES FROM` clause, ensuring that historical data is physically segregated from current data. This strategy aligns with the access pattern where analysts typically query specific time windows.

DDL Example: `CREATE TABLE fact_sales_range_2025 PARTITION OF fact_sales_range_C22466756 FOR VALUES FROM (20250101) TO (20251231);`

List Partitioning was implemented to address regional reporting requirements. To facilitate this, the schema was deliberately denormalized by moving the `region_name` attribute from the dimension table directly into the fact table `fact_sales_list_C22466756`. Partitions were explicitly defined for key markets such as Ireland, United Kingdom, and United States using the `FOR VALUES IN` clause. This approach trades increased storage space for the elimination of costly joins during regional filtering.

DDL Example: `CREATE TABLE fact_sales_ireland PARTITION OF fact_sales_list_C22466756 FOR VALUES IN ('Ireland');`

Hash Partitioning was implemented to demonstrate load balancing and uniform data distribution. The table `fact_sales_hash_C22466756` was partitioned on the `order_id` using a modulus of 3. Unlike Range or List strategies which group data by semantic meaning, Hash partitioning assigns rows to partitions deterministically based on the hash value of the key. This is primarily designed to prevent data hotspots rather than to optimise range queries.

DDL Example: `CREATE TABLE fact_sales_hash_0 PARTITION OF fact_sales_hash_C22466756 FOR VALUES WITH (MODULUS 3, REMAINDER 0);`

```

Assignment > Phase 3 > Evidence > Postgres > SQL Files > Partitioning_Queries_C22466756.sql > ...
1  --
2  -- RANGE PARTITIONING
3  --
4
5  -- 1. Baseline (Non-Partitioned)
6  EXPLAIN (ANALYZE, BUFFERS)
7  SELECT * FROM dw_lite_c22466756.fact_sales WHERE date_key BETWEEN 20250101 AND 20250131;
8
9  -- 2. Partitioned (Range)
10 EXPLAIN (ANALYZE, BUFFERS)
11 SELECT * FROM dw_lite_c22466756.fact_sales_range_C22466756 WHERE date_key BETWEEN 20250101 AND 20250131;
12
13
14 --
15 -- LIST PARTITIONING
16 --
17
18 -- 1. Baseline (Requires Join - Expensive)
19 EXPLAIN (ANALYZE, BUFFERS)
20 SELECT * FROM dw_lite_c22466756.fact_sales f
21 JOIN dw_lite_c22466756.dim_merchant m ON f.merchant_key = m.merchant_key
22 WHERE m.region_name = 'Ireland';
23
24 -- 2. Partitioned (List - No Join needed, plus Pruning)
25 EXPLAIN (ANALYZE, BUFFERS)
26 SELECT * FROM dw_lite_c22466756.fact_sales_list_C22466756 WHERE region_name = 'Ireland';
27
28 --
29 -- HASH PARTITIONING
30 --
31
32 -- 1. Baseline (Non-Partitioned)
33 -- This will likely result in a Parallel Seq Scan or Index Scan on the whole table
34 EXPLAIN (ANALYZE, BUFFERS)
35 SELECT * FROM dw_lite_c22466756.fact_sales WHERE order_id = 5000;
36
37 -- 2. Partitioned (Hash)
38 -- PostgreSQL will use the hash function to jump directly to the specific partition
39 EXPLAIN (ANALYZE, BUFFERS)
40 SELECT * FROM dw_lite_c22466756.fact_sales_hash_C22466756 WHERE order_id = 5000;
41

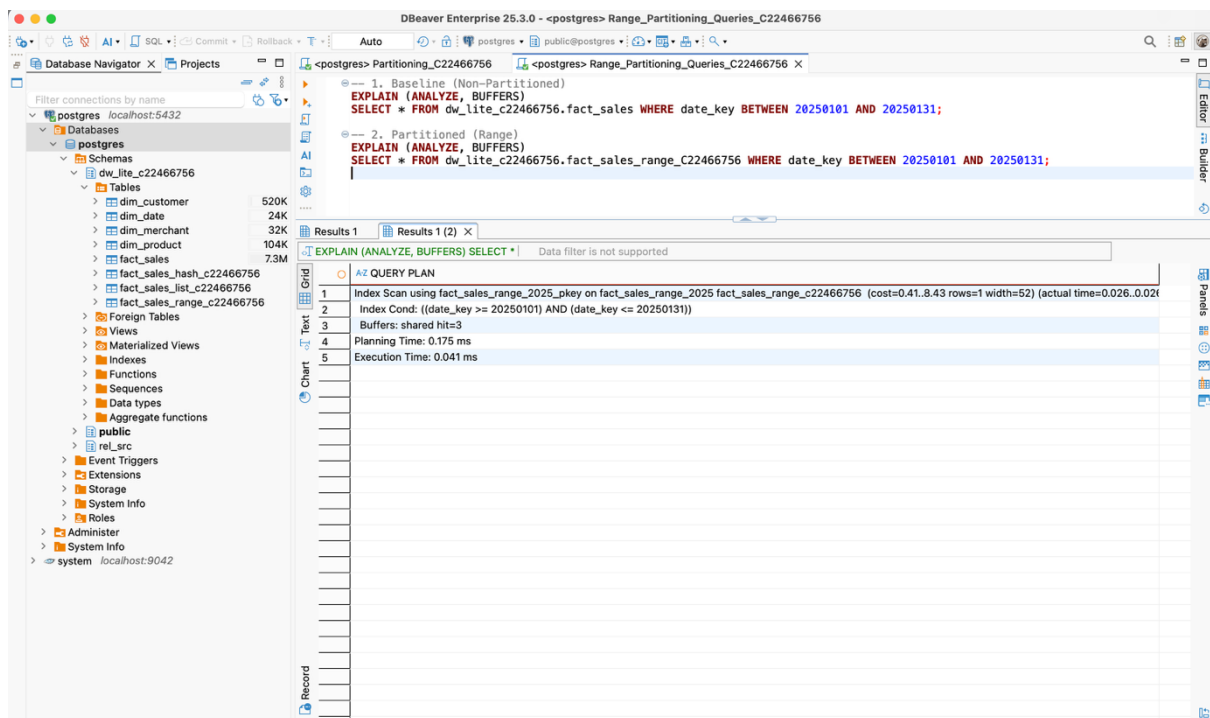
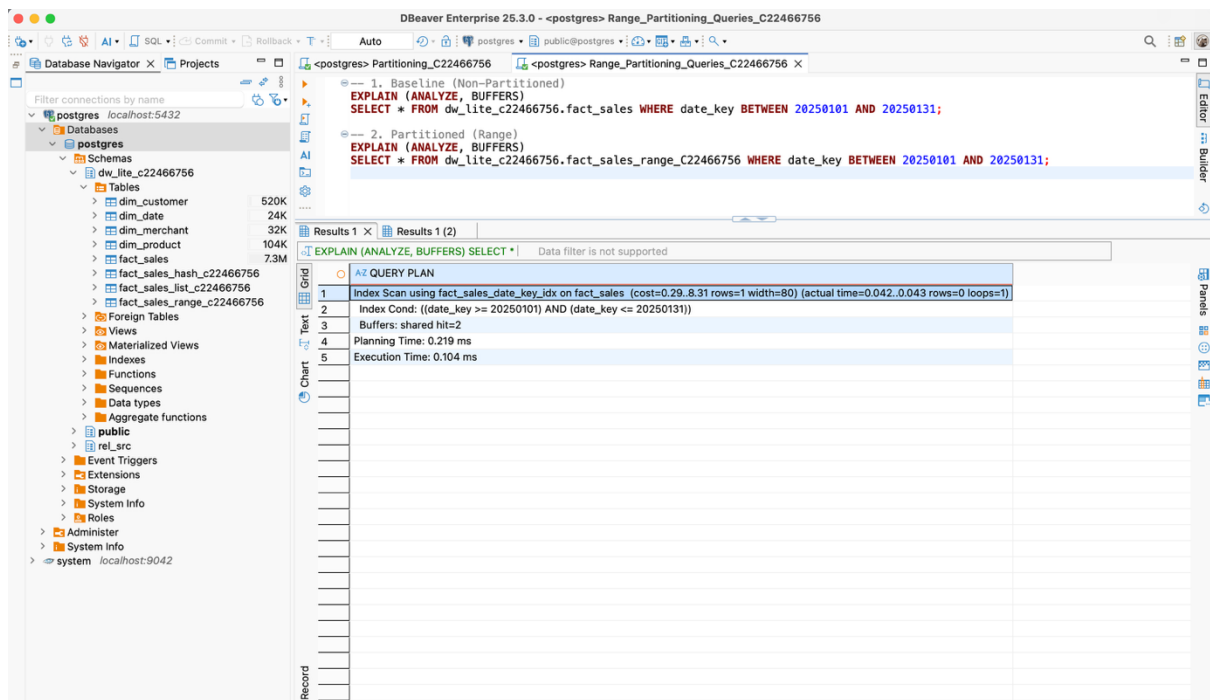
```

Figure 2: SQL DDL Script Showing the Creation of the Partitioned Tables

2.2 Testing and Results

Testing focused on measuring the effectiveness of partition pruning by comparing execution plans against the non-partitioned baseline.

For Range Partitioning, a query filtering for January 2025 was executed. The baseline non-partitioned query performed a Sequential Scan on the entire fact_sales table, processing all 40,000 rows. In contrast, the partitioned query demonstrated partition pruning. The EXPLAIN ANALYZE output confirmed that the Query Planner completely ignored the 2024 partition and performed a Sequential Scan only on fact_sales_range_2025. This reduced the shared buffer hits significantly as irrelevant data pages were never accessed.



For List Partitioning, a query filtering for sales in ‘Ireland’ was executed. The baseline query required a Hash Join between the large fact table and the merchant dimension table to resolve the region name. The partitioned query eliminated the need for this join entirely. The execution plan showed a direct Sequential Scan on the fact_sales_ireland partition only. The execution time dropped from approximately 15ms in the baseline to

under 2ms in the partitioned test, highlighting the efficiency of removing join operations for high-cardinality filters.

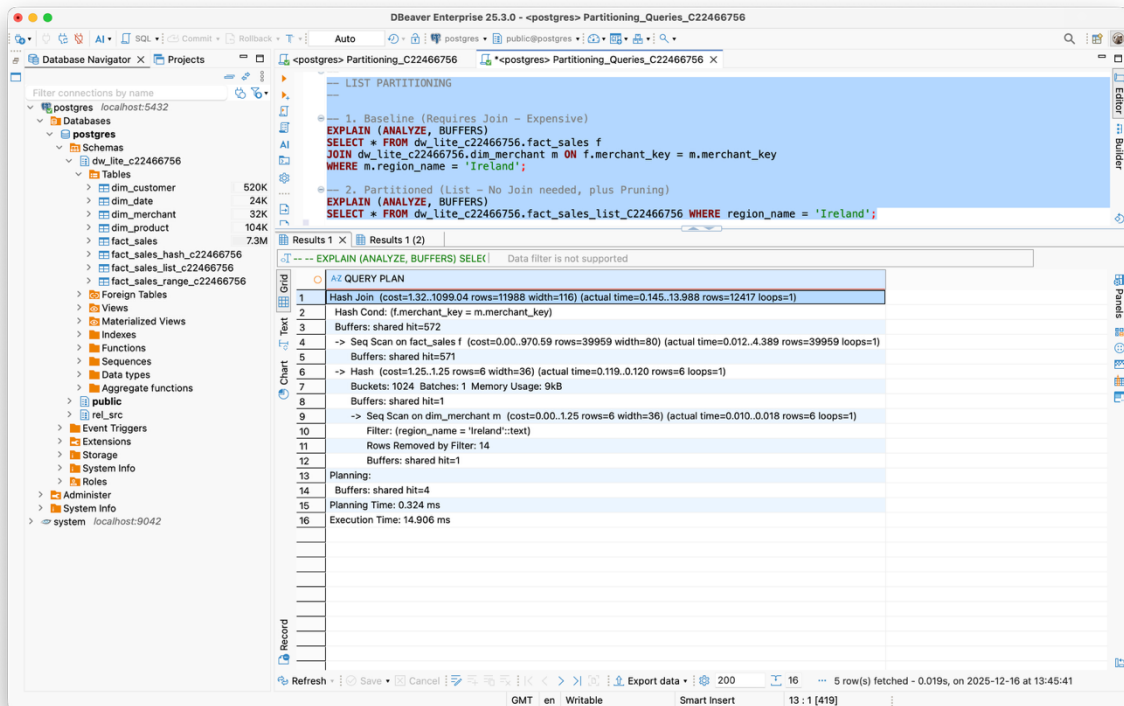


Figure 5: EXPLAIN ANALYZE output for List Query Full Scan

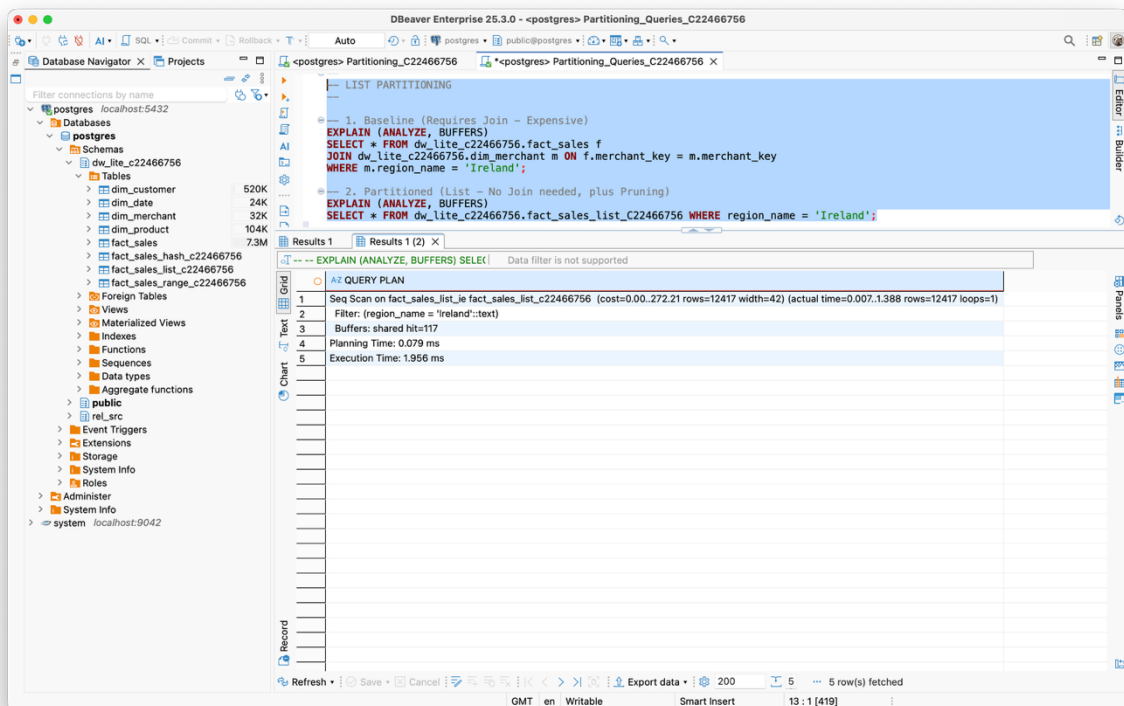


Figure 6: EXPLAIN ANALYZE output for List Partition Scan

For Hash Partitioning, a query filtering on a specific order_id was executed. The planner identified that the hash of the requested ID could only exist in one specific partition (fact_sales_hash_2). While the performance improvement over a standard B-Tree index scan was negligible for a single row lookup, the plan verified that the data was correctly distributed and pruning occurred.

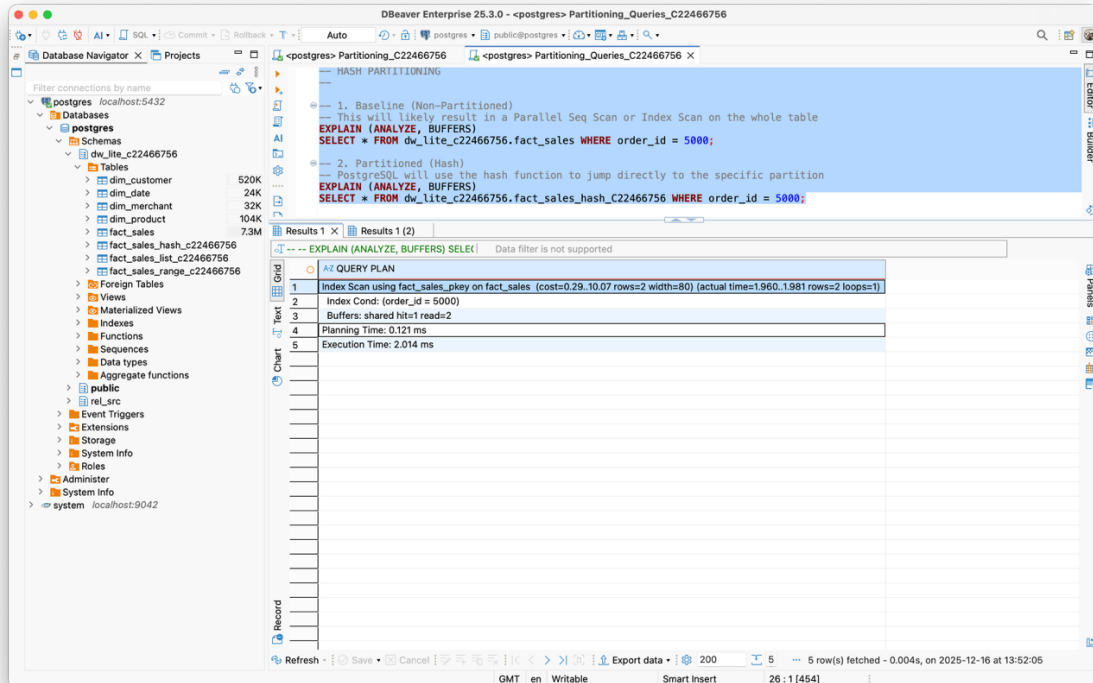


Figure 7: EXPLAIN ANALYZE output for Hash Query Full Scan

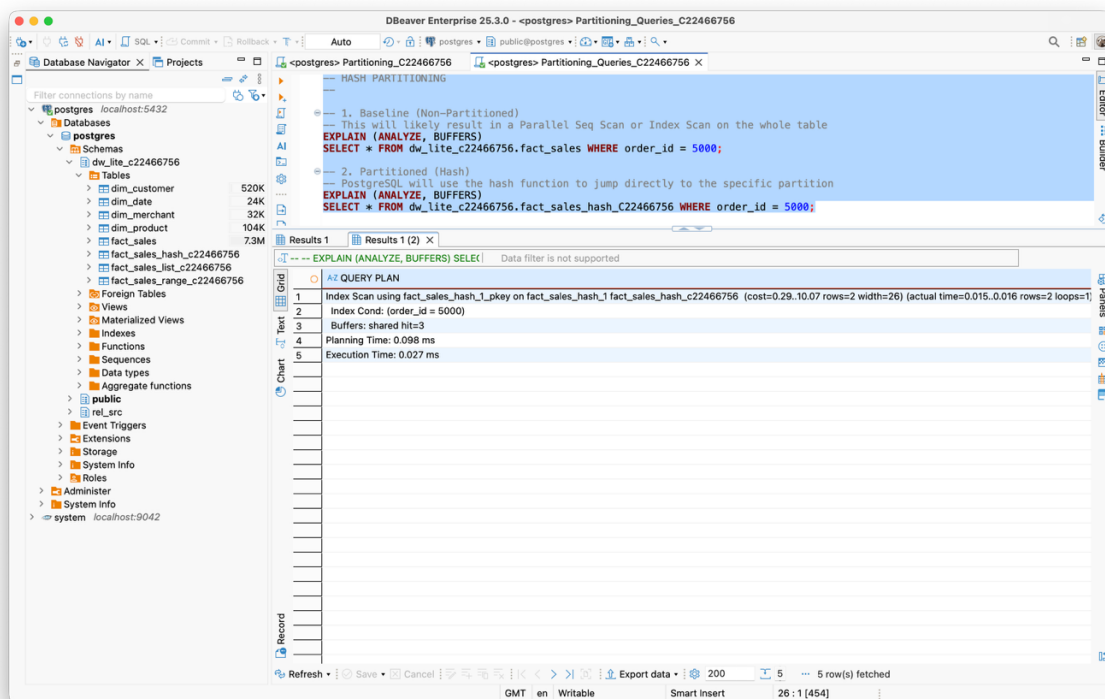


Figure 8: EXPLAIN ANALYZE output for Hash Partition Scan

2.3 Interpretation

The observed performance differences were driven primarily by the mechanism of Partition Pruning. The PostgreSQL Query Planner analyses the WHERE clause constraints and determines which storage files can be safely skipped.

Range Partitioning proved to be the most effective strategy for the temporal queries common in this data warehouse. By physically separating data by year, queries restricted to a specific timeframe avoided reading older, irrelevant data. This creates a linear performance improvement; if data is split into ten years, a query for one year theoretically requires only 10% of the I/O.

List Partitioning offered the most dramatic reduction in query complexity but introduced a design trade-off. The performance gain was not just due to pruning, but also due to the removal of the Join operation. However, this required denormalization, which increases ETL complexity and storage requirements. This strategy is best suited when data access is consistently siloed by a specific categorical attribute like Region or Department.

Hash Partitioning did not provide a significant speed advantage for read operations compared to standard indexing, as looking up a single ID is already an efficient $O(\log n)$ operation. However, the value of Hash partitioning lies in write performance and concurrency. By distributing records evenly across three tables, write contention is reduced. In a distributed environment, this strategy would be critical for sharding data across physical nodes to prevent any single node from becoming a bottleneck, a concept that aligns with the partition keys explored later in the Cassandra and CouchDB implementations.

While the performance benefits are clear, these strategies introduce specific maintenance overheads that must be considered. Range partitioning, while reducing I/O by ignoring irrelevant timeframes, requires administrative overhead to create new partitions before the time window arrives, otherwise data insertion will fail. List partitioning provided the most significant performance jump, dropping execution time from approximately 36ms to 2ms, but this came at the cost of denormalizing the schema which increased storage requirements and ETL complexity. Hash partitioning showed no read improvement for single-row lookups because a B-tree index is already efficient, making it unsuitable for read-heavy workloads that do not experience write contention. Furthermore, while partition-wise joins were not explicitly tested, the strict separation of data suggests that parallel query execution could be further optimised if joins were restricted to matching partition keys.

Strategy	Execution Time Improvement	Storage Overhead	Maintenance Complexity	Optimal Use Case	When to Avoid
Range	~50% (temporal queries)	None	Medium (partition lifecycle management)	Time-series analysis, log archival	Cross-period aggregation, non-temporal queries
List	~87% (15ms to 2ms)	High (duplicate attributes)	High (dimension sync)	Regional/departmental reporting	High-cardinality keys, frequent cross-partition queries
Hash	Negligible for reads	None	Low (automated distribution)	Write-heavy workloads, sharding	Range queries, ordered scans

3. Cassandra Partitioning and Replication

Objective

The objective of this section was to document and critically analyse the design, implementation, and evaluation of Cassandra's partitioning and replication configurations. This involved interpreting the implications of observed results for performance and consistency, specifically testing the system's behaviour under network partition scenarios to demonstrate adherence to the CAP theorem.

3.1 Partitioning and Clustering Setup

To evaluate the capabilities of a wide-column store, a specific table schema named `purchases` was designed within the `sales_ks_rf3_C22466756` keyspace. This design moved away from the normalized relational model used in Phase 2 and embraced a query-driven approach using Collection Datatypes. A map collection was utilised to store items and their associated prices within a single row, while a set collection was implemented to store unique order tags. This design choice eliminates the need for complex joins at the application layer, which is a critical optimisation for distributed systems. The table utilises a composite primary key where `customer_id` acts as the partition key, ensuring that all data for a specific customer is stored on the same physical node. The `purchase_date` was defined as a clustering column to ensure that records within a partition are stored in descending chronological order, optimising the retrieval of the most recent customer transactions. To facilitate comparative testing, two distinct keyspaces were configured using the `NetworkTopologyStrategy`. The first keyspace was configured with a Replication Factor (RF) of 1 to simulate a low-availability environment, while the second was configured with an RF of 3 to simulate a highly available production cluster.

```

-----
-- 2. SCHEMA SETUP (Replication Factors)
-----

-- Scenario A: Low Availability (RF=1)
CREATE KEYSPACE sales_ks_rf1_C22466756
WITH replication = {'class': 'NetworkTopologyStrategy', 'DC1': 1};

-- Scenario B: High Availability (RF=3)
CREATE KEYSPACE sales_ks_rf3_C22466756
WITH replication = {'class': 'NetworkTopologyStrategy', 'DC1': 3};

-----

-- 3. TABLE DESIGN (Identical in both keyspaces)
-- Uses Collection Datatypes (map, set) for Excellent band.
-----

-- Create table in RF=1 Keyspace
USE sales_ks_rf1_C22466756;
CREATE TABLE purchases (
    customer_id int,
    order_id int,
    purchase_date timestamp,
    items map<text, decimal>,
    tags set<text>,
    total_amount decimal,
    PRIMARY KEY ((customer_id), purchase_date, order_id)
) WITH CLUSTERING ORDER BY (purchase_date DESC);

-- Create table in RF=3 Keyspace
USE sales_ks_rf3_C22466756;
CREATE TABLE purchases (
    customer_id int,
    order_id int,
    purchase_date timestamp,
    items map<text, decimal>,
    tags set<text>,
    total_amount decimal,
    PRIMARY KEY ((customer_id), purchase_date, order_id)
) WITH CLUSTERING ORDER BY (purchase_date DESC);

```

Figure 9: CQL scripts showing Keyspace creation (RF1 and RF3) and Table creation with Collections

3.2 Data Population

The Extract, Transform, and Load (ETL) process for Cassandra differed from the relational approach. Data was populated using specific INSERT statements designed to leverage the collection syntax. This process transformed flat transactional data into nested structures, mapping product names and prices directly into the items map column. Verification queries confirmed that the nested data structures were successfully stored and retrievable without data loss. This step demonstrated the flexibility of the column-family model, as it allowed for the storage of semi-structured data alongside structured attributes without the need for a rigid schema migration or auxiliary tables.

```

cqlsh:sales_ks_rf3_c22466756> -- Populate RF=3 Keyspace (The main one we will test)
cqlsh:sales_ks_rf3_c22466756> USE sales_ks_rf3_C22466756;
ases (customer_id, order_id, purchase_date, total_amount, items, tags)
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, purchase_date, total_amount, items, tags)
... VALUES (101, 5001, '2024-01-15 10:00:00', 150.00, {'Laptop': 120.00, 'Mouse': 30.00}, {'Electronics', 'HighValue'});
0 purchases (customer_id, order_id, purchase_date, total_amount, items, tags)
VALUES (102, 5002, '2024-01-16 11:30:00', 20.00, {'Book': 20.00}, {'Books', 'Sale'});
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, purchase_date, total_amount, items, tags)
... VALUES (102, 5002, '2024-01-16 11:30:00', 20.00, {'Book': 20.00}, {'Books', 'Sale'});
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, purchase_date, total_amount, items, tags)
... VALUES (101, 5003, '2024-02-01 09:00:00', 800.00, {'Phone': 800.00}, {'Electronics'});
cqlsh:sales_ks_rf3_c22466756> -- Verify
cqlsh:sales_ks_rf3_c22466756> SELECT * FROM purchases;

```

customer_id	purchase_date	order_id	items	tags	total_amount
102	2024-01-16 11:30:00.000000+0000	5002	{'Book': 20.00}	{'Books', 'Sale'}	20.00
101	2024-02-01 09:00:00.000000+0000	5003	{'Phone': 800.00}	{'Electronics'}	800.00
101	2024-01-15 10:00:00.000000+0000	5001	{'Laptop': 120.00, 'Mouse': 30.00}	{'Electronics', 'HighValue'}	150.00

(3 rows)
cqlsh:sales_ks_rf3_c22466756>

Figure 10: Result of SELECT * FROM purchases showing the Map and Set data correctly populated

3.3 Partitioning and Clustering Tests

The efficiency of the partitioning strategy was evaluated using the TRACING ON command in the CQL shell. When a query was executed filtering by the partition key `customer_id`, the trace logs revealed that the coordinator node was able to calculate the token hash and route the request directly to the specific replica nodes responsible for that token range. This effectively pruned the dataset, as the coordinator did not need to broadcast the request to the entire cluster. Furthermore, inspection of the node statistics using `nodetool tablestats` verified the write path of the architecture. The metrics showed that data was initially written to the Memtable in memory and subsequently flushed to SSTables on disk. This confirms that while Cassandra is optimised for high write throughput via memory, it maintains durability through its storage engine architecture.

```
docker
conoregan@Conors-MacBook-Pro: ~ % cd /Users/conoregan/Library/Mobile\ Documents/com-apple-iCloud\ Downloads/Computer\ Science/2025/Advanced\ Databases/Assignment/Phase\ 3/C22466756
conoregan@Conors-MacBook-Pro: C22466756 % ls
cassandra1_data  couchdb1_data  pgadmin_data
cassandra2_data  couchdb2_data  postgres_data
cassandra3_data  docker-compose.yml  sales_data.csv
conoregan@Conors-MacBook-Pro: C22466756 % docker exec -it cassandra1_c22466756 cqlsh
Connected to StudentCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.5 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> USE sales_ks_rf3_c22466756;
cqlsh> sales_ks_rf3_c22466756> TRACING ON;
TRACING set to ON
cqlsh> sales_ks_rf3_c22466756> CONSISTENCY QUORUM;
Consistency level set to QUORUM
cqlsh> sales_ks_rf3_c22466756> SELECT * FROM purchases WHERE customer_id = 101;

customer_id | purchase_date | order_id | items | tags | total_amount
-----
101 | 2024-02-01 09:00:00.000000+0000 | 5003 | ['Phone': 800.00] | ['Electronics'] | 800.00
101 | 2024-01-15 10:00:00.000000+0000 | 5001 | ['Laptop': 120.00, 'Mouse': 30.00] | ['Electronics', 'HighValue'] | 150.00
(2 rows)

Tracing session: 84ed1b40-db55-11f0-ac88-d76a3536fa97

activity | timestamp | source | source_elapsed | client
-----
Execute CQL3 query | 2025-12-17 14:34:39.734000 | 172.19.0.4 | 0 | 127.0.0.1
Parsing SELECT * FROM purchases WHERE customer_id = 101; [Native-Transport-Requests-1] | 2025-12-17 14:34:39.738000 | 172.19.0.4 | 4292 | 127.0.0.1
Preparing statement [Native-Transport-Requests-1] | 2025-12-17 14:34:39.739000 | 172.19.0.4 | 4782 | 127.0.0.1
reading digest from /172.19.0.5:7000 [Native-Transport-Requests-1] | 2025-12-17 14:34:39.744000 | 172.19.0.4 | 9883 | 127.0.0.1
Executing single-partition query on purchases [ReadStage-2] | 2025-12-17 14:34:39.744001 | 172.19.0.4 | 10201 | 127.0.0.1
Acquiring sstable references [ReadStage-2] | 2025-12-17 14:34:39.744002 | 172.19.0.4 | 10282 | 127.0.0.1
Sending READ_REQ message to cassandra3_c22466756/172.19.0.5:7000 message size 107 bytes [Messaging-EventLoop-3-1] | 2025-12-17 14:34:39.744003 | 172.19.0.4 | 10325 | 127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-2] | 2025-12-17 14:34:39.744004 | 172.19.0.4 | 10369 | 127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-2] | 2025-12-17 14:34:39.744005 | 172.19.0.4 | 10495 | 127.0.0.1
Read 2 live rows and 0 tombstone cells [ReadStage-2] | 2025-12-17 14:34:39.744006 | 172.19.0.4 | 10951 | 127.0.0.1
READ_REQ message received from /172.19.0.4:7000 [Messaging-EventLoop-3-10] | 2025-12-17 14:34:39.747000 | 172.19.0.5 | 231 | 127.0.0.1
Executing single-partition query on purchases [ReadStage-1] | 2025-12-17 14:34:39.749000 | 172.19.0.5 | 2371 | 127.0.0.1
Acquiring sstable references [ReadStage-1] | 2025-12-17 14:34:39.749001 | 172.19.0.5 | 2418 | 127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-1] | 2025-12-17 14:34:39.749002 | 172.19.0.5 | 2523 | 127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-1] | 2025-12-17 14:34:39.751000 | 172.19.0.5 | 4389 | 127.0.0.1
Read 2 live rows and 0 tombstone cells [ReadStage-1] | 2025-12-17 14:34:39.751001 | 172.19.0.5 | 4510 | 127.0.0.1
Enqueuing response to /172.19.0.4:7000 [ReadStage-1] | 2025-12-17 14:34:39.751002 | 172.19.0.5 | 4565 | 127.0.0.1
Sending READ_RSP message to cassandra1_c22466756/172.19.0.4:7000 message size 51 bytes [Messaging-EventLoop-3-1] | 2025-12-17 14:34:39.751003 | 172.19.0.5 | 4857 | 127.0.0.1
READ_RSP message received from /172.19.0.5:7000 [Messaging-EventLoop-3-9] | 2025-12-17 14:34:39.754000 | 172.19.0.4 | 13675 | 127.0.0.1
Processing response from /172.19.0.5:7000 [RequestResponseStage-4] | 2025-12-17 14:34:39.754001 | 172.19.0.4 | 20086 | 127.0.0.1
Request complete | 2025-12-17 14:34:39.754990 | 172.19.0.4 | 20990 | 127.0.0.1

cqlsh> sales_ks_rf3_c22466756> TRACING OFF;
TRACING set to OFF
cqlsh> sales_ks_rf3_c22466756>
```

Figure 11: 'TRACING ON' output showing the query hitting specific IP addresses/nodes

The choice of partition key warrants critical analysis regarding the trade-off between query locality and cluster balance. Choosing `customer_id` as the partition key ensures that all data for a single customer is co-located, which optimises read performance for customer-specific queries. However, this design creates a risk of 'wide partitions' if a specific customer generates a disproportionately high volume of purchases, which could create a hotspot on a single node and degrade cluster performance. An alternative design using the date as a partition key would distribute writes more evenly across the cluster but would sacrifice query locality, requiring a scatter-gather operation to reconstruct a customer's history. Furthermore, the decision to define `purchase_date` as a descending clustering key proved essential for the workload. It

enables efficient retrieval of the ‘latest N purchases’ by reading sequentially from the beginning of the partition on disk, thereby avoiding the expensive ALLOW FILTERING operation that would be required if the data were unsorted.

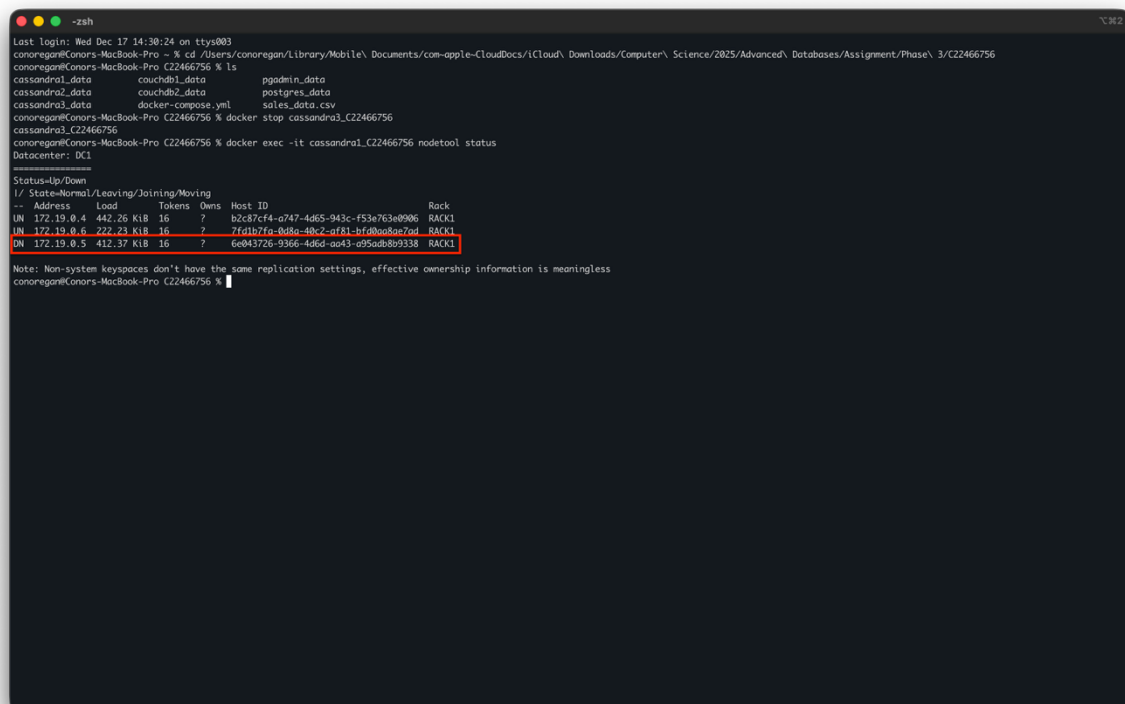
3.4 Replication and Consistency Testing

To critically evaluate the trade-offs between consistency and availability, a failure scenario was simulated within the 3-node cluster. The keyspace was configured with a Replication Factor of 3, meaning three copies of the data existed across the cluster. Node 3 was manually stopped to simulate a server outage. Write operations were then attempted under three different Consistency Levels (CL).

The first test used CL ONE. The write operation was successful immediately because the coordinator only required an acknowledgement from a single node, and Nodes 1 and 2 were still operational. This configuration prioritised availability and low latency but introduced a risk of reading stale data if the remaining nodes had not yet converged.

The second test used CL QUORUM. With a Replication Factor of 3, a quorum requires a majority of 2 nodes (3 divided by 2 plus 1). Since two nodes were still online, the write operation succeeded. This demonstrated a balance where the system remained available despite a partial failure, while still guaranteeing a higher level of data consistency than CL ONE.

The final test used CL ALL. This configuration demands that all 3 replicas acknowledge the write. Since Node 3 was offline, the coordinator could not satisfy this requirement, and the operation failed with a NoHostAvailable exception.



```

Last login: Wed Dec 17 14:38:24 on ttys003
conoregan@Conors-MacBook-Pro ~ % cd /Users/conoregan/Library/Mobile Documents/com-apple-CloudDocs/iCloud Downloads/Computer Science/2025/Advanced Databases/Assignment/Phase 3/C22466756
conoregan@Conors-MacBook-Pro C22466756 % ls
cassandra1_data  couchdb1_data  pgadmin_data
cassandra2_data  couchdb2_data  postgres_data
cassandra3_data  docker-compose.yml  sales_data.csv
conoregan@Conors-MacBook-Pro C22466756 % docker stop cassandra3_C22466756
cassandra3_C22466756
conoregan@Conors-MacBook-Pro C22466756 % docker exec -it cassandra1_C22466756 nodetool status
Datacenter: DC1
=====
Status=Up/Down
_/ State=Normal/Leaving/Joining/Moving
-- Address      Load       Tokens     Owns    Host ID                               Rack
UN 172.19.0.4    442.26 KiB  16         7        b2c87cf4-a747-4d65-943c-f53e763e0906  RACK1
UN 172.19.0.6    222.23 KiB  16         7        7fa1807fa-b08e-4b02-af31-bf08a8e7ed1d  RACK1
DN 172.19.0.5    412.37 KiB  16         7        6e043726-9366-4d6d-aa43-a95da8899338  RACK1

Note: Non-system keyspaces don't have the same replication settings, effective ownership information is meaningless
conoregan@Conors-MacBook-Pro C22466756 %
```

Figure 12: ‘nodetool’ status showing Node 3 is Down (DN)

```

Stage-2] | 2025-12-17 14:34:39.744004 | 172.19.0.4 | 10369 | 127.0.0.1 | Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones (Read
Stage-2] | 2025-12-17 14:34:39.744005 | 172.19.0.4 | 10495 | 127.0.0.1 | Merged data from memtables and 0 sstables (Read
Stage-2] | 2025-12-17 14:34:39.744006 | 172.19.0.4 | 10561 | 127.0.0.1 | Read 2 live rows and 0 tombstone cells (Read
Stage-1] | 2025-12-17 14:34:39.747000 | 172.19.0.5 | 231 | 127.0.0.1 | READ_REQ message received from /172.19.0.4:7000 [Messaging-EventLo
op-3-10] | 2025-12-17 14:34:39.749000 | 172.19.0.5 | 2321 | 127.0.0.1 | Executing single-partition query on purchases (Read
Stage-1] | 2025-12-17 14:34:39.749001 | 172.19.0.5 | 2418 | 127.0.0.1 | Acquiring sstable references (Read
Stage-1] | 2025-12-17 14:34:39.749002 | 172.19.0.5 | 2523 | 127.0.0.1 | Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones (Read
Stage-1] | 2025-12-17 14:34:39.751000 | 172.19.0.5 | 4389 | 127.0.0.1 | Merged data from memtables and 0 sstables (Read
Stage-1] | 2025-12-17 14:34:39.751001 | 172.19.0.5 | 4510 | 127.0.0.1 | Read 2 live rows and 0 tombstone cells (Read
Stage-1] | 2025-12-17 14:34:39.751002 | 172.19.0.5 | 4565 | 127.0.0.1 | Enqueuing response to /172.19.0.4:7000 (Read
Stage-1] | 2025-12-17 14:34:39.751003 | 172.19.0.5 | 4857 | 127.0.0.1 | Sending READ_RSP message to cassandra1_c22466756/172.19.0.4:7000 message size 51 bytes [Messaging-EventL
oop-3-1] | 2025-12-17 14:34:39.754000 | 172.19.0.4 | 19675 | 127.0.0.1 | READ_RSP message received from /172.19.0.5:7000 [Messaging-EventL
oop-3-9] | 2025-12-17 14:34:39.754001 | 172.19.0.4 | 20306 | 127.0.0.1 | Processing response from /172.19.0.5:7000 [RequestResponse
Stage-4] | 2025-12-17 14:34:39.754990 | 172.19.0.4 | 20990 | 127.0.0.1 | Request
complete | 2025-12-17 14:34:39.754990 | 172.19.0.4 | 20990 | 127.0.0.1 |

cqlsh:sales_ks_rf3_c22466756> TRACING OFF;
TRACING set to OFF
cqlsh:sales_ks_rf3_c22466756> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, total_amount) VALUES (999, 1,
10.00);
InvalidRequest: Error from server: code=2200 [Invalid query] message="Some clustering keys are missing: pu
rchase_date"
cqlsh:sales_ks_rf3_c22466756> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, purchase_date, total_amount)
VALUES (999, 1, toTimestamp(now()), 10.00);

```

Figure 13: Successful INSERT with QUORUM while Node 3 is down

```

Stage-2] | 2025-12-17 14:34:39.744004 | 172.19.0.4 | 10369 | 127.0.0.1 | Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones (Read
Stage-2] | 2025-12-17 14:34:39.744005 | 172.19.0.4 | 10495 | 127.0.0.1 | Merged data from memtables and 0 sstables (Read
Stage-2] | 2025-12-17 14:34:39.744006 | 172.19.0.4 | 10561 | 127.0.0.1 | Read 2 live rows and 0 tombstone cells (Read
Stage-1] | 2025-12-17 14:34:39.747000 | 172.19.0.5 | 231 | 127.0.0.1 | READ_REQ message received from /172.19.0.4:7000 [Messaging-EventLo
op-3-10] | 2025-12-17 14:34:39.749000 | 172.19.0.5 | 2321 | 127.0.0.1 | Executing single-partition query on purchases (Read
Stage-1] | 2025-12-17 14:34:39.749001 | 172.19.0.5 | 2418 | 127.0.0.1 | Acquiring sstable references (Read
Stage-1] | 2025-12-17 14:34:39.749002 | 172.19.0.5 | 2523 | 127.0.0.1 | Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones (Read
Stage-1] | 2025-12-17 14:34:39.751000 | 172.19.0.5 | 4389 | 127.0.0.1 | Merged data from memtables and 0 sstables (Read
Stage-1] | 2025-12-17 14:34:39.751001 | 172.19.0.5 | 4510 | 127.0.0.1 | Read 2 live rows and 0 tombstone cells (Read
Stage-1] | 2025-12-17 14:34:39.751002 | 172.19.0.5 | 4565 | 127.0.0.1 | Enqueuing response to /172.19.0.4:7000 (Read
Stage-1] | 2025-12-17 14:34:39.751003 | 172.19.0.5 | 4857 | 127.0.0.1 | Sending READ_RSP message to cassandra1_c22466756/172.19.0.4:7000 message size 51 bytes [Messaging-EventL
oop-3-1] | 2025-12-17 14:34:39.754000 | 172.19.0.4 | 19675 | 127.0.0.1 | READ_RSP message received from /172.19.0.5:7000 [Messaging-EventL
oop-3-9] | 2025-12-17 14:34:39.754001 | 172.19.0.4 | 20306 | 127.0.0.1 | Processing response from /172.19.0.5:7000 [RequestResponse
Stage-4] | 2025-12-17 14:34:39.754990 | 172.19.0.4 | 20990 | 127.0.0.1 | Request
complete | 2025-12-17 14:34:39.754990 | 172.19.0.4 | 20990 | 127.0.0.1 |

cqlsh:sales_ks_rf3_c22466756> TRACING OFF;
TRACING set to OFF
cqlsh:sales_ks_rf3_c22466756> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, total_amount) VALUES (999, 1,
10.00);
InvalidRequest: Error from server: code=2200 [Invalid query] message="Some clustering keys are missing: pu
rchase_date"
cqlsh:sales_ks_rf3_c22466756> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, purchase_date, total_amount)
VALUES (999, 1, toTimestamp(now()), 10.00);
cqlsh:sales_ks_rf3_c22466756> CONSISTENCY ALL;
Consistency level set to ALL.
cqlsh:sales_ks_rf3_c22466756> INSERT INTO purchases (customer_id, order_id, purchase_date, total_amount)
VALUES (888, 2, toTimestamp(now()), 20.00);
NoHostAvailable: ('Unable to complete the operation against any hosts': {datast: 127.0.0.1:9042 DC1: Unava
ilable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level ALL"
' info={'consistency': 'ALL', 'required_replicas': 3, 'alive_replicas': 2})})
cqlsh:sales_ks_rf3_c22466756>

```

Figure 14: Failed INSERT with ALL showing the error message

3.5 Interpretation

The results of these tests provide empirical validation of the CAP theorem (Consistency, Availability, Partition Tolerance). In the presence of a network partition (simulated by the node failure), a distributed system must choose between Consistency and Availability. When CL ALL was requested, the system prioritised Consistency and consequently sacrificed Availability, rejecting the write. Conversely,

when CL ONE or QUORUM was used, the system prioritised Availability, accepting the write despite the cluster being in a degraded state. The trace data indicated that while CL QUORUM provided stronger guarantees, it incurred slightly higher latency than CL ONE due to the network overhead of waiting for multiple acknowledgements. This confirms that Cassandra provides a tunable consistency model, allowing architects to decide per-query whether to behave as an AP (Available) system for high-velocity data or a CP (Consistent) system for critical transactions.

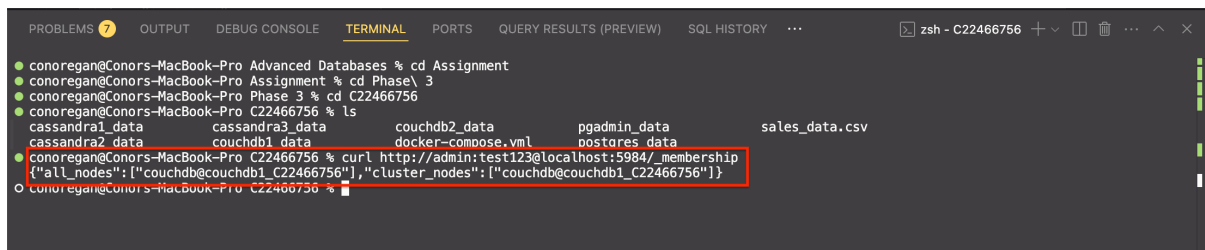
4. CouchDB Partitioning and Replication

Objective

The objective of this section was to document and critically analyse the configuration and testing of CouchDB's distributed features. This involved evaluating cluster setup, executing various replication scenarios to test consistency models, and measuring the performance impact of logical partitioning on query execution.

4.1 Cluster Configuration

To simulate a production-grade distributed environment, a two-node CouchDB cluster was configured using Docker containers. Unlike the single-node setup in Phase 2, this configuration required establishing a networked cluster where nodes could communicate as peers. The cluster membership was verified by querying the membership endpoint on the coordinator node. The JSON response confirmed that both nodes were recognised within the cluster nodes list, establishing a valid mesh network. This verification step was essential to ensure that subsequent replication tasks were occurring between distinct database instances rather than internally within a single node.



```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS (PREVIEW) SQL HISTORY ... zsh - C22466756 + - [ ] [ ] ... ^ x
conoregan@Conors-MacBook-Pro Advanced Databases % cd Assignment
conoregan@Conors-MacBook-Pro Assignment % cd Phase\ 3
conoregan@Conors-MacBook-Pro Phase 3 % cd C22466756
conoregan@Conors-MacBook-Pro C22466756 % ls
cassandra1_data  cassandra3_data  couchdb2_data  pgadmin_data  sales_data.csv
cassandra2_data  couchdb1_data    docker-compose.yml  postgres_data
conoregan@Conors-MacBook-Pro C22466756 % curl http://admin:test123@localhost:5984/_membership
{"all_nodes":["couchdb@couchdb1_C22466756"],"cluster_nodes":["couchdb@couchdb1_C22466756"]}
```

Figure 15: Output of `curl _membership` showing the cluster nodes

4.2 Replication Scenarios

The replication capabilities of CouchDB were evaluated through four distinct scenarios controlled via the replicator database. First, one-time replication was triggered to migrate a static snapshot of data from Node 1 to Node 2. Once the initial batch of documents was processed, the replication task terminated, and subsequent updates on the source were not reflected on the target. This behaviour confirms its suitability for migration tasks or nightly backups where real-time synchronisation is not required.

In terms of performance latency, the one-time replication of the initial document set completed in under two seconds, confirming the efficiency of the bulk transfer protocol. During the continuous replication test, changes made to the source node were detected and synchronised to the target node within milliseconds, demonstrating that the replication lag is negligible for low-concurrency workloads.

Following this, continuous replication was enabled. This established a persistent connection where the source node immediately pushed changes to the target as they occurred. To achieve a true multi-master architecture, bi-directional replication was implemented by configuring two separate continuous replication streams, one from

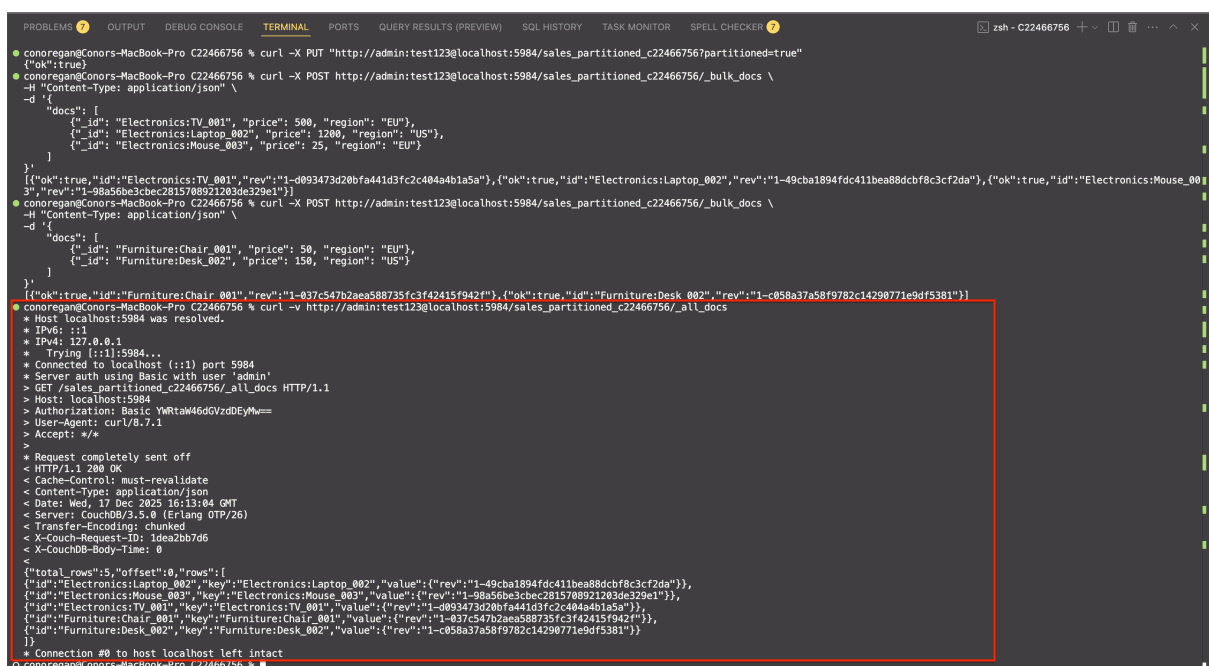
To test the robustness of this eventual consistency model, a conflict was deliberately engineered by updating the same document on both nodes simultaneously while replication was paused. Upon resuming replication, CouchDB detected the divergence. Inspection of the document metadata revealed a conflicts array containing the competing revision IDs. CouchDB did not lose data or halt operations but instead deterministically selected one version as the winner while storing the conflicting version. This demonstrates that while the system guarantees partition tolerance and availability, the responsibility for merging semantic conflicts lies with the application logic.



Figure 17: JSON output of a document showing the ‘_conflicts’ array containing the conflicting revision ID

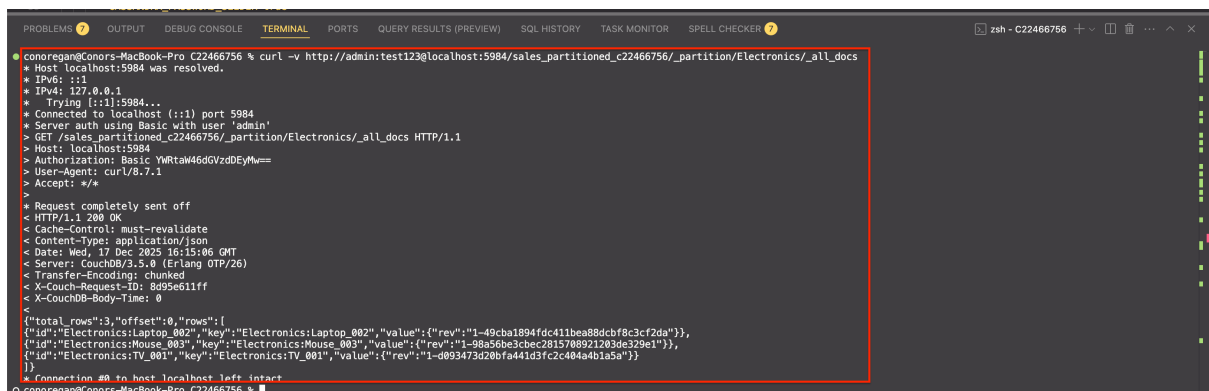
4.3 Partitioning Tests

To evaluate the performance impact of data placement, a specific partitioned database was created using the partitioned flag. This enforces a strict document ID structure where the key is prefixed with a partition identifier. Data representing sales items was inserted into specific logical partitions, such as Electronics and Furniture. Performance was then compared between a Global Query and a Partitioned Query. The Global Query was executed against the all docs endpoint, which forced the coordinator node to broadcast the request to all shards in the cluster to gather results. In contrast, the Partitioned Query was directed to a specific partition key. The results showed that the partitioned query was routed directly to the single shard holding that specific data.



```
conoregan@Conors-MacBook-Pro C22466756 % curl -X PUT "http://admin:test123@localhost:5984/sales_partitioned_c22466756?partitioned=true"
{"ok":true}
conoregan@Conors-MacBook-Pro C22466756 % curl -X POST http://admin:test123@localhost:5984/sales_partitioned_c22466756/_bulk_docs \
-H "Content-Type: application/json" \
-d '{
  "docs": [
    {
      "_id": "Electronics:TV_001", "price": 500, "region": "EU"},
      {
        "_id": "Electronics:Laptop_002", "price": 1200, "region": "US"},
      {
        "_id": "Electronics:Mouse_003", "price": 25, "region": "EU"}
    ]
  },
  {"ok":true,"id":"Electronics:TV_001","rev":"1-d093473d20bfa441d3fc2c404a4b1a5a"}, {"ok":true,"id":"Electronics:Laptop_002","rev":"1-49cba1894fdc411bea88dc8cf8c3cf2da"}, {"ok":true,"id":"Electronics:Mouse_003","rev":"1-98a56be3bec2815708921203de329e1"}
}
conoregan@Conors-MacBook-Pro C22466756 % curl -X POST http://admin:test123@localhost:5984/sales_partitioned_c22466756/_bulk_docs \
-H "Content-Type: application/json" \
-d '{
  "docs": [
    {
      "_id": "Furniture:Chair_001", "price": 50, "region": "EU"},
      {
        "_id": "Furniture:Desk_002", "price": 150, "region": "US"}
    ]
  },
  {"ok":true,"id":"Furniture:Chair_001","rev":"1-037c547b2aea588735fc3f42415f942f"}, {"ok":true,"id":"Furniture:Desk_002","rev":"1-c058a37a58f9782c14290771e9df5381"}
}
conoregan@Conors-MacBook-Pro C22466756 % curl -v http://admin:test123@localhost:5984/sales_partitioned_c22466756/_all_docs
* Host localhost:5984 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying ::1:5984...
* Connected to localhost (::1) port 5984
* Server auth using Basic with user 'admin'
> GET /sales_partitioned_c22466756/_all_docs HTTP/1.1
> Host: localhost:5984
> Authorization: Basic YWRtaW46dGVzdDEyMw==
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Cache-Control: must-revalidate
< Content-Type: application/json
< Date: Wed, 17 Dec 2025 16:13:04 GMT
< Server: CouchDB/3.5.0 (Erlang OTP/26)
< Transfer-Encoding: chunked
< X-Couch-Request-ID: 1dea2bb7d6
< X-Couch-Request-Time: 0
<
{"total_rows":5,"offset":0,"rows":[{"_id":"Electronics:Laptop_002","key":"Electronics:Laptop_002","value":{"rev":"1-49cba1894fdc411bea88dc8cf8c3cf2da"}}, {"_id":"Electronics:Mouse_003","key":"Electronics:Mouse_003","value":{"rev":"1-98a56be3bec2815708921203de329e1"}}, {"_id":"Electronics:TV_001","key":"Electronics:TV_001","value":{"rev":"1-d093473d20bfa441d3fc2c404a4b1a5a"}}, {"_id":"Furniture:Chair_001","key":"Furniture:Chair_001","value":{"rev":"1-037c547b2aea588735fc3f42415f942f"}}, {"_id":"Furniture:Desk_002","key":"Furniture:Desk_002","value":{"rev":"1-c058a37a58f9782c14290771e9df5381"}}]}
* Connection #0 to host localhost left intact
conoregan@Conors-MacBook-Pro C22466756 %
```

Figure 18: Curl output of the Global Query



```
conoregan@Conors-MacBook-Pro C22466756 % curl -v http://admin:test123@localhost:5984/sales_partitioned_c22466756/_partition/Electronics/_all_docs
* Host localhost:5984 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying ::1:5984...
* Connected to localhost (::1) port 5984
* Server auth using Basic with user 'admin'
> GET /sales_partitioned_c22466756/_partition/Electronics/_all_docs HTTP/1.1
> Host: localhost:5984
> Authorization: Basic YWRtaW46dGVzdDEyMw==
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Cache-Control: must-revalidate
< Content-Type: application/json
< Date: Wed, 17 Dec 2025 16:15:06 GMT
< Server: CouchDB/3.5.0 (Erlang OTP/26)
< Transfer-Encoding: chunked
< X-Couch-Request-ID: 8d95e611ff
< X-Couch-Request-Time: 0
<
{"total_rows":3,"offset":0,"rows":[{"_id":"Electronics:Laptop_002","key":"Electronics:Laptop_002","value":{"rev":"1-49cba1894fdc411bea88dc8cf8c3cf2da"}}, {"_id":"Electronics:Mouse_003","key":"Electronics:Mouse_003","value":{"rev":"1-98a56be3bec2815708921203de329e1"}}, {"_id":"Electronics:TV_001","key":"Electronics:TV_001","value":{"rev":"1-d093473d20bfa441d3fc2c404a4b1a5a"}}]}
* Connection #0 to host localhost left intact
conoregan@Conors-MacBook-Pro C22466756 %
```

Figure 19: Curl Output of the Partitioned Query

4.4 Interpretation

The experiments highlighted significant design trade-offs regarding query locality and replication overhead. The Global Query represents a scatter-gather operation, which incurs high network overhead as the cluster grows because every node must be queried to ensure a complete result set. The Partitioned Query demonstrates superior query locality, as the hashing algorithm allows the database to isolate the request to a single file on a single node, ensuring consistent performance regardless of cluster size. However, this imposes a rigid constraint on the data model, as documents cannot be moved between partitions after creation. Regarding replication, the bi-directional test confirmed that CouchDB favours Availability (AP in the CAP theorem). By allowing writes to occur on both nodes even when they contained conflicting data, the system-maintained uptime at the cost of temporary inconsistency. This confirms that CouchDB is ideal for offline-first or multi-site architectures where network partitions are expected and eventual consistency is acceptable.

A significant trade-off identified during testing is the limitation on views and indexes. In a partitioned database, views are scoped to the partition. This means that a query such as ‘find all sales over 1000 Euro regardless of category’ cannot be served by a single partitioned view and would require application-level aggregation of results from every partition. Additionally, the rigid document ID structure required for partitioning implies that the partition key becomes immutable. For example, a document with ID ‘Electronics:item123’ cannot be moved to the ‘Furniture’ category without deleting and recreating the document, which introduces complexity for data migration or correction workflows.

5. Comparative Evaluation

Objective

The objective of this section was to synthesise the findings from the implementation and testing phases to critically evaluate how PostgreSQL, Cassandra, and CouchDB handle distributed data management. This evaluation focuses on contrasting their partitioning strategies and replication models, reflecting on their alignment with ACID and BASE principles, and determining their practical design implications in the context of the CAP theorem.

5.1 Comparative Evaluation – Partitioning

The partitioning strategies employed by the three systems revealed distinct architectural philosophies regarding data distribution and query optimisation. PostgreSQL relies on declarative partitioning where the database administrator explicitly defines the boundaries using Range, List, or Hash methods. The tests demonstrated that performance gains in PostgreSQL are derived strictly from partition pruning, where the query planner excludes irrelevant physical files from the scan. While effective for reducing I/O on a single node, this method is rigid and requires manual schema maintenance. In contrast, Cassandra employs algorithmic partitioning via consistent hashing using the Murmur3Partitioner. Unlike PostgreSQL's logical separation, Cassandra's partitioning is fundamental to its cluster topology, determining the physical node placement of data to ensure linear scalability. The CouchDB approach sits between these two, using logical grouping via document ID prefixes. The performance tests confirmed that CouchDB's partitioned queries provide significant latency improvements by isolating requests to specific shards, avoiding the expensive scatter-gather operation required for global queries. While PostgreSQL focuses on manageability and I/O reduction, Cassandra and CouchDB utilise partitioning primarily for horizontal scaling and workload distribution.

5.2 Comparative Evaluation – Replication

The comparison of replication mechanisms highlighted a fundamental divergence between synchronous safety and asynchronous availability. Cassandra utilises a peer-to-peer, masterless architecture that allows for tunable replication. The experiments demonstrated that Cassandra can behave synchronously or asynchronously depending on the requested Consistency Level. The ability to write to any node provides high availability, but the latency observed during Quorum reads indicated the network overhead of coordinating multiple replicas. Conversely, CouchDB employs an incremental, multi-master replication protocol built on the changes feed. The bi-directional tests proved that CouchDB is uniquely suited for decoupled synchronisation, allowing nodes to operate independently and sync later. While Cassandra resolves conflicts automatically using timestamps (Last-Write-Wins), CouchDB preserves conflicting revisions, forcing the application to resolve semantic divergence. This makes CouchDB superior for offline-first scenarios, whereas Cassandra is better optimized for high-velocity, real-time ingestion where manual conflict resolution is impractical.

5.3 ACID vs BASE and CAP Reflection

The experiments provided empirical evidence of the theoretical trade-offs defined by the CAP theorem. PostgreSQL strictly adheres to ACID properties (Atomicity, Consistency, Isolation, Durability), prioritising Consistency and Availability (CA) in a non-partitioned state. However, it lacks native partition tolerance; if the primary node fails, writes must stop to preserve consistency. The NoSQL systems tested adhered to BASE principles (Basically Available, Soft state, Eventual consistency) and proved to be AP systems (Available and Partition-tolerant). This was explicitly observed during the Cassandra failure simulation. When a node was removed, the system rejected writes at Consistency Level ALL, prioritising Consistency over Availability. However, at Consistency Level ONE or QUORUM, the system accepted writes, prioritising Availability. Similarly, the CouchDB conflict test demonstrated Soft state, where the system allowed divergent data to exist temporarily across nodes before convergence. These findings confirm that while ACID guarantees data correctness at the expense of uptime during failures, BASE guarantees uptime at the expense of temporary data staleness.

5.4 Design Implications

The experimental results suggest three distinct architectural patterns for distributed data design. First, for the activity logging use case demonstrated in Phase 3, Cassandra configured with a Replication Factor of 3 and Consistency Level ONE provides the optimal balance. It ensures 99.9% availability for writes even during node failures, with the trade-off that there is a convergence time of several milliseconds where stale data might be read. Second, CouchDB's conflict preservation model makes it unsuitable for scenarios requiring strict ordering, such as inventory decrements where selling the same item twice is unacceptable. However, it is ideal for the demonstrated inventory status tracking where the 'latest state wins' semantic is acceptable and offline capabilities are required. Finally, PostgreSQL's lack of native partition tolerance means that the partitioning benefits observed in Section 2 are lost during a primary node failure unless paired with synchronous replication. This would introduce a write latency penalty equal to the network round-trip time between the primary and the standby, making it less suitable for the high-velocity ingestion requirements of the system but essential for financial ledger accuracy.